

Documentación Técnica Pokémon Battle Simulator

Proyecto de Desarrollo Web con Laravel y Consumo de API

Índice

- 1. Visión General del Proyecto
 - Objetivos Técnicos
- 2. Arquitectura del Sistema
 - Stack Tecnológico
 - Estructura de Directorios Clave
- 3. Lógica de Negocio y Servicios
 - 3.1. Gestión de Datos (Fachada y Capas)
 - 3.2. Sistema de Batalla
 - 3.3. Inteligencia Artificial (AIService)
 - 3.4. Características Avanzadas (Pokedex y Objetos)
- 4. Estructura de Base de Datos
- 5. Frontend (Blade & UI)
- 6. Puntos Críticos y Futuras Mejoras
- 7. USO DE IA EN EL PROYECTO
 - Herramientas y Modelos Utilizados
 - Descripción de Uso
 - Prompts Significativos
- 8. REFLEXIÓN PERSONAL
 - Experiencia de Programar con IA
 - Dificultades y Ventajas Encontradas
 - Influencia de la IA en el Futuro como Desarrollador

1. Visión General del Proyecto

Este proyecto es una aplicación web desarrollada con el framework **Laravel 11** (PHP) que simula un sistema de combates Pokémon completo. Actúa como un motor de juego basado en navegador que integra el consumo de una API externa (PokéAPI), lógica matemática precisa para el cálculo de daño y estadísticas, y una interfaz de usuario temática "Sci-Fi/Terminal".

La aplicación permite a los usuarios gestionar un equipo de Pokémon (**TeamController**), consultar información detallada en una Pokédex (**PokemonController**) y combatir contra una Inteligencia Artificial (PvE) o contra otro jugador en modo local (PvP) (**BattleController**).

Objetivos Técnicos

El proyecto ha sido diseñado para demostrar competencias avanzadas en desarrollo Full Stack:

1. **Arquitectura Híbrida de Datos (Cache-First):** Implementación de un sistema de triple capa para la obtención de datos: Memoria Caché -> Base de Datos Relacional -> API Externa. Esto optimiza drásticamente el rendimiento y reduce la dependencia de servicios externos.
2. **Patrones de Diseño:** Uso extensivo de **Facades** (Fachadas) para simplificar el acceso a datos complejos y **Services** (Servicios) para desacoplar la lógica de negocio de los controladores.
3. **Simulación Matemática:** Recreación fiel de las fórmulas de daño, cálculo de estadísticas por nivel y mecánicas de tipos de los videojuegos originales de Pokémon.
4. **Normalización de Base de Datos:** Transición de un esquema simple basado en JSON a un modelo relacional robusto (3FN) para garantizar la integridad de los datos.

2. Arquitectura del Sistema

El sistema sigue el patrón **MVC (Modelo-Vista-Controlador)**, pero enriquecido con capas adicionales de Servicios y Repositorios para mantener el código limpio y mantenible (Clean Code). Esta estructura modular permite escalar el proyecto sin romper funcionalidades existentes.

Stack Tecnológico

- **Backend:** Laravel 11.
- **Lenguaje:** PHP 8.2+.
- **Frontend:** Blade Templates, JavaScript ES6 (Vanilla) y CSS3 (Variables CSS, Grid, Flexbox).
- **Base de Datos:** MySQL.
- **Cliente HTTP:** Guzzle (para peticiones a PokéAPI).
- **Autenticación:** Laravel Sanctum (preparado para API).

Instalación del cliente HTTP

```
./vendor/bin/sail composer require guzzlehttp/guzzle
```

Estructura de Directorios Clave y Responsabilidades

```

app/
├─ Helpers/
│   └─ PokemonHelper.php      # [FACHADA] Punto único de acceso a datos.
Orquesta la caché, BD y API.
├─ Http/
│   └─ Controllers/
│       └─ BattleController.php # Gestiona el flujo HTTP de la batalla
(inicio, ataque, cambio).
│       └─ PokemonController.php # Gestiona la visualización de la Pokedex
y listas.
│       └─ TeamController.php   # Gestiona la creación y edición del
equipo del usuario.
├─ Repositories/
│   └─ PokemonRepository.php   # Capa encargada exclusivamente de hablar
con la Base de Datos.
├─ Services/                  # La "Lógica del Negocio". Donde ocurren los
cálculos reales.
│   └─ AIService.php           # Cerebro de la máquina para elegir ataques
estratégicos.
│   └─ BattleService.php       # Árbitro que controla turnos, reglas y
validaciones.
│   └─ BattleCalculator.php     # Matemático que calcula el daño exacto con
fórmulas oficiales.
│   └─ BattleLogService.php     # Narrador que escribe qué pasó en el turno.
│   └─ ItemService.php         # Farmacéutico que gestiona inventarios y
uso de pociones.
│   └─ PokeApiService.php      # Mensajero que descarga datos de la API
externa.
│   └─ StatusEffectService.php # Médico que revisa si un Pokémon está
quemado o dormido.

```

3. Lógica de Negocio y Servicios

3.1. Gestión de Datos (Fachada y Capas)

Para evitar que la aplicación sea lenta consultando siempre a internet, usamos un patrón de **Fachada** (**PokemonHelper**). Funciona como una ventanilla única:

1. **Caché (Memoria)**: Pregunta primero a Redis o Archivo. Recuperar datos aquí toma milisegundos.
2. **Base de Datos (Repository)**: Si no está en memoria, **PokemonRepository** busca en el disco duro local usando SQL optimizado.
3. **API Externa (Service)**: Si es un Pokémon nuevo (ej. "Mewtwo"), se conecta a la PokéAPI, descarga los datos crudos, los **normaliza** (limpia JSONs anidados, mapea estadísticas) y los guarda en nuestra BD transaccionalmente para que la próxima vez sea instantáneo.

3.2. Sistema de Batalla

El combate es el corazón del proyecto. Varios servicios trabajan en equipo coordinados por **BattleController**:

- **BattleService (El Árbitro)**: Mantiene la matriz de tipos (18x18). Sabe que *Agua* vence a *Fuego* (x2) y *Eléctrico* no afecta a *Tierra* (x0). Valida si un Pokémon puede atacar o si ha sido derrotado (HP <= 0).
- **BattleCalculator (El Matemático)**: Aplica la fórmula oficial de daño de Nivel 50:

Daño = $((2 * \text{Nivel} / 5 + 2) * \text{Potencia} * A / D) / 50 + 2$ * Modificadores
 Los modificadores incluyen **STAB** (Bonus por mismo tipo), **Efectividad**, **Críticos** (1/16 de probabilidad) y un **Factor Aleatorio** (0.85 - 1.00) para realismo.

- **StatusEffectService (El Médico)**: Procesa efectos al final del turno.
 - *Quemadura (Burn)*: Resta 1/16 de vida y reduce el ataque físico a la mitad.
 - *Parálisis (Paralysis)*: Reduce velocidad y tiene 25% de probabilidad de impedir el ataque.

3.3. Inteligencia Artificial (AIService)

La máquina no elige al azar (a menos que sea nivel Fácil). Evalúa el tablero para tomar decisiones simulando un jugador humano:

- **Easy**: Elige cualquier movimiento disponible al azar usando `array_rand()`.
- **Normal**: Calcula el daño potencial de sus 4 movimientos y elige el que cause más daño en ese turno específico.
- **Hard**: Piensa tácticamente.
 - *Supervivencia*: Si tiene < 50% de vida y pociones, se cura.
 - *Cambio Estratégico*: Si su tipo es desventajoso (Planta contra tu Fuego), busca en su equipo un Pokémon resistente (ej. Agua) y cambia.
 - *Debuffs*: Intenta aplicar estados (quemar/envenenar) al inicio del combate si el rival está sano.

3.4. Características Avanzadas (Pokedex y Objetos)

- **Bio-Acústica (Cries):** Integración multimedia real. Al ver un Pokémon, `PokemonController` recupera la URL de su "grito" digitalizado de la tabla `pokemon_cries` para reproducirlo.
- **Gestión de Inventario:** `ItemService` maneja un sistema de objetos persistente en sesión. Permite usar Pociones (curan HP), Revivir (resucitan estado fainted) o Curas Totales (eliminan burn/poison/etc), afectando instantáneamente el estado del combate.

4. Estructura de Base de Datos

No guardamos "json" desordenado. Hemos **normalizado** la base de datos a **Tercera Forma Normal (3NF)** para asegurar integridad y velocidad:

Tabla	Descripción	Detalle Técnico
<code>pokemons</code>	Tabla Madre.	ID, nombre, peso, altura, exp_base. Sin datos repetidos.
<code>types & stats</code>	Catálogos.	Listas maestras de tipos (Fuego, Agua) y stats (Ataque, Velocidad).
<code>pokemon_types</code>	Pivote (Relación).	Vincula Pokémon con Tipos. Columna <code>slot</code> define tipo primario/secundario.
<code>pokemon_moves</code>	Pivote (Relación).	Relación compleja. Guarda <code>level_learned_at</code> para saber cuándo aprende cada ataque.
<code>pokemon_sprites</code>	Multimedia.	URLs de imágenes (Front, Back, Shiny) separadas para no ensuciar la tabla principal.

Esta estructura, gestionada por migraciones de Laravel, permite consultas SQL potentes como *"Dame todos los movimientos de tipo Dragón que aprende Charizard"*.

5. Frontend (Blade & UI)

La interfaz es lo que conecta al usuario con toda esta lógica compleja.

- **Estética "Silph Co.":** Inspirada en interfaces de ciencia ficción/terminal. Uso de paleta de colores oscuros (#1a1a1a) con acentos neón (Cyan #00f3ff, Verde #00ff9d) para resaltar información crítica sin cansar la vista.
- **Team Management (Command Center):** Panel táctico en `team/index.blade.php`. Permite reordenar el equipo y visualizar estadísticas clave con barras de progreso visuales.
- **Battle Arena Reactiva:**
 - Las barras de salud (HP) bajan suavemente usando transiciones CSS (`transition: width 0.5s`).
 - Colores semánticos: Verde (>50%), Amarillo (>20%), Rojo Crítico (<20%).
 - Animaciones CSS keyframes para efectos de daño (parpadeo rojo) y flotación de sprites.

6. Puntos Críticos y Futuras Mejoras

1. **Multijugador Real (WebSockets con Laravel Reverb):**

Actualmente el PvP es local. La evolución natural es implementar **Laravel Reverb** (el nuevo servidor WebSocket nativo de Laravel 11). Esto permitiría canales privados seguros para que dos jugadores se envíen eventos de "Ataque Seleccionado" en tiempo real, manteniendo todo el stack en PHP.

2. **Optimización con Colas (Laravel Jobs):**

La carga inicial de datos desde PokéAPI puede ser lenta. Podemos implementar **Jobs y Queues** de Laravel para procesar estas importaciones en segundo plano (background), mejorando la respuesta del servidor y evitando timeouts.

3. **Evolución del Frontend (Laravel Livewire / Inertia):**

Para mantenernos en el ecosistema Laravel pero modernizar la UI, el siguiente paso lógico es **Livewire**

3. Nos permitiría tener componentes reactivos (como las barras de vida o el chat de batalla) escribiendo solo código PHP y Blade, sin la complejidad de separar una API para React/Vue.

7. USO DE IA EN EL PROYECTO

Herramientas y Modelos Utilizados

Para el desarrollo de este proyecto se han utilizado diversas herramientas de Inteligencia Artificial como **DeepSeek**, **GitHub Copilot**, **ChatGPT**, **Claude** y, de manera destacada, **Gemini (Google DeepMind)** como asistente principal para arquitectura y lógica compleja.

Descripción de Uso

La IA no solo "escribió código", sino que ayudó a pensar actuando como un Senior Developer acompañante:

- **Arquitecto de Software:** Ayudó a diseñar la estructura de carpetas y decidir el uso de Patrones de Diseño (Repository para BD, Service para Lógica, Facade para Acceso).
- **Matemático del Juego:** Explicó y tradujo a código PHP las complejas fórmulas de daño y efectividad de tipos (Type Matchups) de los juegos originales.
- **Diseñador UI:** Generó ideas y código CSS para lograr el efecto de "brillo de neón", gradientes y las animaciones fluidas de las barras de vida.
- **Debugger:** Ayudó a encontrar errores difíciles, como problemas de asincronía en turnos o cálculos de daño con valores negativos.

Prompts Significativos

Ejemplos de cómo se interactuó con la IA para obtener resultados precisos:

- *"Analiza la clase BattleService y dime si hay lógica repetida que pueda extraer a una función privada para limpiar el código."*
- *"Explicame cómo funciona la tabla de tipos de Pokémon y ayúdame a crear una migración SQL para representarla eficientemente."*
- *"Crea un diseño CSS Grid para una Pokedex que parezca una terminal futurista, usando colores oscuros, fuentes monoespaciadas y bordes cían."*

8. REFLEXIÓN PERSONAL

Experiencia de Programar con IA

Programar asistido por IA ha transformado el flujo de trabajo de "escribir código" a "orquestar soluciones". De ser quien escribe el código, al que supervisa el código como si fuera el jefe del proyecto, supervisando el trabajo de los integrantes del equipo. La experiencia ha sido genial, permitiendo abordar una lógica de batalla compleja, logrando una imitación de las mecánicas oficiales de Pokémon. Trabajando con IA, he logrado resolver problemas complejos en una fracción del tiempo que hubiera tomado manualmente. La IA actúa como un compañero de programación (Pair Programmer) siempre disponible, capaz de sugerir alternativas, detectar errores sutiles y explicar conceptos oscuros de frameworks o librerías. No todo ha sido perfecto, ya que a veces la IA propone soluciones que no son las más óptimas o que requieren de una explicación adicional para entenderlas. Y ha veces la IA propone soluciones que no son las más acertadas, las cuales destruyen la funcionalidad del proyecto, eliminan cosas que eran útiles o funcionaban. Gracias al control de versiones, he podido revertir los cambios y continuar con el proyecto. Y seguir aprendiendo a usar la IA en el desarrollo de software.

Dificultades y Ventajas Encontradas

Ventajas:

- **Velocidad de Iteración:** La capacidad de generar prototipos funcionales de UI o lógica de backend en minutos.
- **Reducción de Carga Cognitiva:** Delegar la memorización de sintaxis específica o configuraciones repetitivas permite concentrarse en la arquitectura y la experiencia de usuario.
- **Aprendizaje Continuo:** La IA sugiere patrones de diseño (como Repositorios o Servicios) que elevan la calidad del código y sirven como herramienta educativa.

Dificultades:

- **Pérdida de Contexto:** En sesiones largas o proyectos grandes, la IA a veces "olvida" decisiones de diseño anteriores o correcciones aplicadas, reintroduciendo bugs antiguos.
- **Alucinaciones:** Ocasionalmente sugiere métodos de Laravel o CSS que no existen o están deprecados, requiriendo verificación constante.
- **Dependencia en Depuración:** A veces es tentador pedir a la IA que arregle un error sin entenderlo completamente, lo que puede llevar a soluciones "parche" en lugar de arreglos de raíz.

Influencia de la IA en el Futuro como Desarrollador

La integración de la IA en el desarrollo de software parece inevitable y positiva. En mi futuro como desarrollador, veo a la IA no como un reemplazo, sino como un multiplicador de fuerza. El rol evolucionará de ser un "codificador" puro a un perfil más arquitectónico y de revisión, donde la habilidad clave será saber *qué* pedir, *cómo* evaluar la calidad del código generado y *cómo* integrar piezas complejas en un sistema coherente. La IA permitirá construir software más ambicioso, robusto y creativo, eliminando las barreras de entrada técnicas para materializar ideas complejas.